C++ COMPLETE REFERENCE LIBRARY • MODULE 10

10 INHERITANCE — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 inheritance fundamentals and access modes
3. **Theory Chapter:** 02 mandatory classroom examples analysis
4. **Theory Chapter:** 03 types of inheritance and constructor order
5. **Theory Chapter:** 04 ambiguity diamond problem and virtual base
6. **Visual Memory & Concept Diagrams** (3 Vector Diagrams)
7. **Annotated C++ Source Programs** (8 Programs)
8. **Practice Question Bank:** conceptual questions
9. **Practice Question Bank:** debugging questions
10. **Practice Question Bank:** predict the output
11. **Practice Question Bank:** programming questions
12. **Step-by-Step Solutions:** conceptual solutions
13. **Step-by-Step Solutions:** output solutions
14. **Step-by-Step Solutions:** programming solutions

10 — Inheritance (Priority Deep-Dive Module)

This module provides a complete, classroom-aligned treatment of C++ Inheritance, including the three inheritance visibility modes (`public` , `protected` , `private`), detailed analysis of your two mandatory classroom `protected` -inheritance examples (`super` / `sub` and `person` / `student`), all five forms of inheritance hierarchies, constructor/destructor execution order, multiple inheritance ambiguity, and solving the Diamond Problem using `virtual` base classes.

Directory Contents

1. Theory (`01_THEORY/`)

- `01_inheritance_fundamentals_and_access_modes.md` (./01_THEORY/01_inheritance_fundamentals_and_access_modes.md) — Base and derived classes, `public` , `protected` , and `private` inheritance access conversion matrix.
- `02_mandatory_classroom_examples_analysis.md` (./01_THEORY/02_mandatory_classroom_examples_analysis.md) — **Deep-Dive on Mandatory Classroom**

Examples 1 & 2 (`super` / `sub` and `person` / `student`), inside-derived vs. outside-derived (`main()`) accessibility rules, and compiler diagnostics.

- **`03_types_of_inheritance_and_constructor_order.md`** (./01_THEORY/03_types_of_inheritance_and_constructor_order.md) — Single, multilevel, multiple, hierarchical, and hybrid inheritance; constructor/destructor call order and function overriding.
- **`04_ambiguity_diamond_problem_and_virtual_base.md`** (./01_THEORY/04_ambiguity_diamond_problem_and_virtual_base.md) — Scope resolution for ambiguity (`B1::show()`), the Diamond Problem subobject duplication, and `virtual` base classes (`class D : virtual public B`).

2. Programs (`02_PROGRAMS/`)

- **`01_single_public_inheritance.cpp`** (./02_PROGRAMS/01_single_public_inheritance.cpp) — Category A: Simple classroom `person` and `student` with `public` inheritance.
- **`02_classroom_super_sub_protected_compile_error.cpp`** (./02_PROGRAMS/02_classroom_super_sub_protected_compile_error.cpp) — Category B: **Mandatory Classroom Example 1** (`obj.show()` intentional compilation error under `protected` inheritance).
- **`03_classroom_super_sub_fixed_public.cpp`** (./02_PROGRAMS/03_classroom_super_sub_fixed_public.cpp) — Category A: Corrected Classroom Example 1 using `public` inheritance.
- **`04_classroom_super_sub_fixed_wrapper.cpp`** (./02_PROGRAMS/04_classroom_super_sub_fixed_wrapper.cpp) — Category A: Corrected Classroom Example 1 keeping `protected` inheritance and adding a `public` member function inside `sub` .
- **`05_classroom_person_student_compile_error.cpp`** (./02_PROGRAMS/05_classroom_person_student_compile_error.cpp) — Category B: **Mandatory Classroom Example 2** (`s.input(40)` fails in `main()` , while `display()` calling `input(30)` succeeds).
- **`06_classroom_person_student_corrected.cpp`** (./02_PROGRAMS/06_classroom_person_student_corrected.cpp) — Category A: Corrected Classroom Example 2 demonstrating valid `s.display()` execution.
- **`07_multilevel_and_constructor_order.cpp`** (./02_PROGRAMS/07_multilevel_and_constructor_order.cpp) — Category A: Constructor and destructor invocation order in a 3-level hierarchy.
- **`08_diamond_problem_virtual_base.cpp`** (./02_PROGRAMS/08_diamond_problem_virtual_base.cpp) — Category A: Solving the Diamond Problem with `virtual public` inheritance.

3. Practice Questions (`03_PRACTICE_QUESTIONS/`)

- **`conceptual_questions.md`** (./03_PRACTICE_QUESTIONS/conceptual_questions.md)
- **`predict_the_output.md`** (./03_PRACTICE_QUESTIONS/predict_the_output.md)
- **`debugging_questions.md`** (./03_PRACTICE_QUESTIONS/debugging_questions.md)
- **`programming_questions.md`** (./03_PRACTICE_QUESTIONS/programming_questions.md)

4. Detailed Solutions (`04_DETAILED_SOLUTIONS/`)

- `conceptual_solutions.md` (./04_DETAILED_SOLUTIONS/conceptual_solutions.md)
- `output_solutions.md` (./04_DETAILED_SOLUTIONS/output_solutions.md)
- `programming_solutions.md` (./04_DETAILED_SOLUTIONS/programming_solutions.md)

5. Technical Diagrams (`05_DIAGRAMS/`)

- `diagram_01_access_specifier_transformation.svg`
  (./05_DIAGRAMS/diagram_01_access_specifier_transformation.svg)
- `diagram_02_classroom_protected_inheritance.svg`
  (./05_DIAGRAMS/diagram_02_classroom_protected_inheritance.svg)
- `diagram_03_inheritance_types_and_diamond.svg`
  (./05_DIAGRAMS/diagram_03_inheritance_types_and_diamond.svg)

01 — Inheritance Fundamentals and Access Specifier Transformation

Inheritance Access Specifier Transformation Matrix

Base Member	: public Base	: protected Base	: private Base
public	public	protected	private
protected	protected	protected	private
private	Inaccessible	Inaccessible	Inaccessible

Access Specifier Transformation

1. Definition & Intuition

Inheritance is the object-oriented mechanism by which a new class (called the **derived class** or **sub class** or **child class**) acquires the data members and member functions of an existing class (called the **base class** or **super class** or **parent class**), while adding its own specialized members or overriding existing behavior.

Real-World Analogy

Every `student` **is a** `person`. A `person` has `age` and `name`. Instead of rewriting `age` and `name` inside `student`, `employee`, and `teacher`, we define `person` once as the **base class** and let `student` inherit from `person`.

2. Syntax of Inheritance

CPP

```cpp
class base_class {
    // base members
};

class derived_class : access_mode base_class {
    // derived members
};
```

- `access_mode` (also called the *base-specifier*) can be `public`, `protected`, or `private`.
- **Default Inheritance Rule**: If `access_mode` is omitted:

- For a `class` (`class sub : super`), the default inheritance mode is **`private`**.
- For a `struct` (`struct sub : super`), the default inheritance mode is **`public`**.

---

3. The Three Member Access Specifiers vs. The Three Inheritance Modes

Never confuse **where a member is declared in the base class** with **how the derived class inherits the base class**!

A. Base Class Member Access Specifiers

1. **`private` in Base**: Accessible **only** inside member functions (and `friend`s) of the Base class itself. Even the Derived class **cannot** directly touch `private` members of Base! (Note: the `private` data members of Base *are* still physically present inside every Derived object in memory; they just can only be accessed via `public` or `protected` functions of Base.)
2. **`protected` in Base**: Accessible inside Base **and** inside member functions of Derived classes, but **inaccessible** from outside code (such as `main()`).
3. **`public` in Base**: Accessible everywhere (unless restricted by `protected` or `private` inheritance in the derived class).

B. Master Access Conversion Table (`[class.access.base]`)

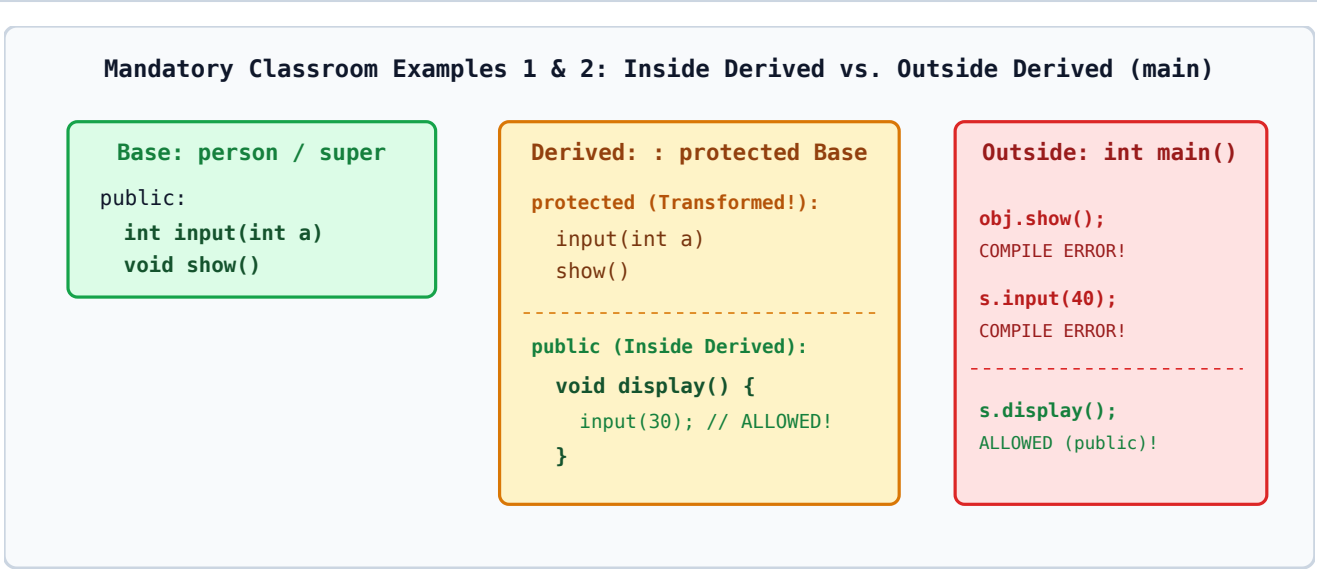
Base Class Member Access	`class Sub : public Super`	`class Sub : protected Super`	`class Sub : private Super`
`public` in `Super`	Becomes **`public`** in `Sub`	Becomes **`protected`** in `Sub`	Becomes **`private`** in `Sub`
`protected` in `Super`	Becomes **`protected`** in `Sub`	Becomes **`protected`** in `Sub`	Becomes **`private`** in `Sub`
`private` in `Super`	Not directly accessible in `Sub`	Not directly accessible in `Sub`	Not directly accessible in `Sub`

---

4. Who Can Access Inherited Members Through What? (Exam Golden Rule)

Effective Access of Inherited Member in `Sub`	Can `Sub`'s own member functions call it?	Can a further derived class `GrandSub : public Sub` call it?	Can `main()` call it via `Sub obj; obj.member()`?
`public` in `Sub`	**YES**	**YES**	**YES**
`protected` in `Sub`	**YES**	**YES**	**NO (Compile Error!)**
`private` in `Sub`	**YES**	**NO (Compile Error!)**	**NO (Compile Error!)**
(Originally `private` in `Super`)	**NO (Compile Error!)**	**NO (Compile Error!)**	**NO (Compile Error!)**

02 — Deep-Dive Analysis: Mandatory Classroom Examples 1 and 2



Classroom Protected Inheritance Diagram

1. Mandatory Classroom Example 1: `super` and `sub` with `protected` Inheritance

Original Classroom Program (Intentional Compilation Error)

CPP

```cpp
#include <iostream>
using namespace std;

class super {
public:
    void show() {
        cout << "hello from super" << endl;
    }
};

class sub : protected super {
};

int main() {
    sub obj;
    obj.show(); // Intentional compilation error
    return 0;
}
```

Detailed Step-by-Step Technical Analysis

1. **In `class super`**: The member function `void show()` is declared under the `public:` access specifier. If we created a `super` object (`super s; s.show();`), that call would be completely legal.

2. **In `class sub : protected super`**: The inheritance mode is `protected`. According to C++ Standard `[class.access.base]`, when a base class is inherited as `protected`, every `public` member of the base class (`super::show()`) is transformed into a **`protected` member of the derived class (`sub`)**.
3. **In `int main()`**:
   - `main()` is a standalone non-member function outside `class sub`.
   - When `main()` executes `obj.show();` on `sub obj;`, name lookup finds `super::show()` through `sub`.
   - Because `show()` is **`protected` inside `sub`**, only member functions of `sub`, friends of `sub`, or classes further derived from `sub` are permitted to access it.
   - `main()` is **outside** the class hierarchy, so the compiler rejects `obj.show();` at compile time!

Typical Compiler Diagnostic (GCC / Clang)

```text
error: 'void super::show()' is inaccessible within this context
note: declared here: void show()
error: 'super' is not an accessible base of 'sub'
```

Two Ways to Fix Classroom Example 1

- **Fix Approach A (Change Inheritance Mode to `public`)**:

  ```cpp
  class sub : public super { };
  ```

  Now `super::show()` remains `public` in `sub`, so `obj.show();` in `main()` compiles and outputs `hello from super`. (See `03_classroom_super_sub_fixed_public.cpp` (../02_PROGRAMS/03_classroom_super_sub_fixed_public.cpp)).

- **Fix Approach B (Keep `protected` Inheritance, Call `show()` from a `public` Member Function of `sub`)**:

  ```cpp
  class sub : protected super {
  public:
    void call_show() {
        show(); // LEGAL! Inside sub, protected inherited member show() is accessible!
    }
  };
  ```

  Then in `main()`, call `obj.call_show();`. (See `04_classroom_super_sub_fixed_wrapper.cpp` (../02_PROGRAMS/04_classroom_super_sub_fixed_wrapper.cpp)).

---

2. Mandatory Classroom Example 2: `person` and `student` (Inside Derived vs. Outside Derived)

Original Classroom Program (Intentional Compilation Error on `s.input(40)`)

CPP

```cpp
#include <iostream>
using namespace std;

class person {
public:
    int age;

    int input(int a) {
        age = a;
        return age;
    }
};

class student : protected person {
public:
    void display() {
        cout << input(30) << endl;
    }
};

int main() {
    student s;

    s.input(40); // Intentional compilation error

    s.display();
    return 0;
}
```

Crucial Conceptual Distinction: Inside Derived (`display()`) vs. Outside Derived (`main()`)

Students often ask: *"If `student` inherits `person` as `protected`, why does `input(30)` work inside `display()`, whereas `s.input(40)` fails inside `main()`?"*

Let us trace the **Access Transformation** of every member of `person` when inherited by `class student : protected person`:

Member in `person`	Original Access in `person`	Inheritance Mode in `student`	Effective Access in `student`	Accessible Inside `student::display()`?	Accessible Outside in `main()` via `s`?
`int age`	`public`	`protected`	**`protected`**	**YES** (Inside derived class)	**NO** (Compile Error!)
`int input(int a)`	`public`	`protected`	**`protected`**	**YES** (Inside derived class)	**NO** (Compile Error!)
`void display()`	*(Defined in `student`)*	N/A	**`public`**	**YES**	**YES** (`s.display()` is valid!)

Why `display()` Can Call `input(30)`

- `display()` is a **member function of** `class student`.
- Inside the body of `student::display()`, the code has full access to all `public`, `protected`, and `private` members of `student`.
- Because `person::input(int)` became a **`protected` member of** `student`, `student::display()` is completely authorized to call `input(30)` (which sets `this→age = 30` and returns `30`).

Why `s.input(40)` Fails in `main()`

- `main()` is **outside** `class student`.
- Outside code can **only** access `public` members of `student` (`s.display()`).
- Because `input(int)` is `protected` inside `student`, writing `s.input(40);` in `main()` violates C++ access control (`[class.access.inh]`) and triggers a compile-time error!

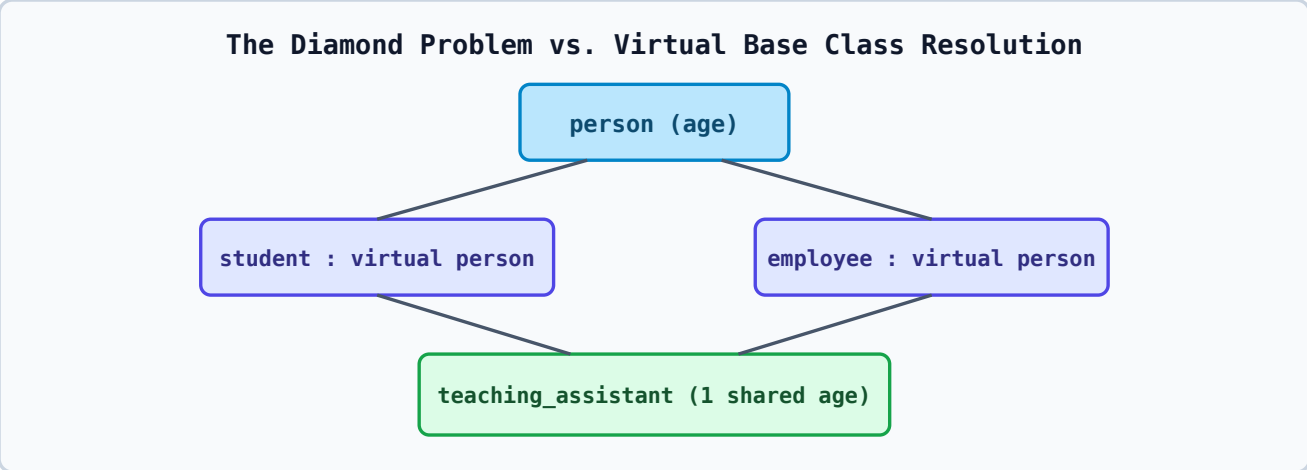
Corrected Version of Classroom Example 2

When the illegal line `s.input(40);` is removed (or commented out) in `main()`, leaving `s.display();`, the program compiles cleanly and prints:

```text
30
```

(See `05_classroom_person_student_compile_error.cpp` (../02_PROGRAMS/05_classroom_person_student_compile_error.cpp) and `06_classroom_person_student_corrected.cpp` (../02_PROGRAMS/06_classroom_person_student_corrected.cpp)).

03 — Five Types of Inheritance, Constructor/Destructor Order, and Overriding



Inheritance Types and Diamond

1. The Five Types of Inheritance in C++

1. **Single Inheritance**: One derived class inherits from a single base class (`A → B`).

```cpp
class person { ... };
class student : public person { ... };
```

2. **Multilevel Inheritance**: A derived class acts as a base class for another derived class in a chain (`A → B → C`).

```cpp
class person { ... };
class student : public person { ... };
class grad_student : public student { ... };
```

3. **Multiple Inheritance**: A single derived class inherits directly from two or more base classes (`A, B → C`).

```cpp
class teacher { ... };
class researcher { ... };
class professor : public teacher, public researcher { ... };
```

4. **Hierarchical Inheritance**: Multiple derived classes inherit from a single common base class (`A → B` , `A → C` , `A → D`).

```cpp
class shape { ... };
class circle : public shape { ... };
class rectangle : public shape { ... };
```

5. **Hybrid (Multipath) Inheritance**: A combination of two or more types of inheritance (for example, Hierarchical + Multiple inheritance forming a **Diamond** structure).

---

2. Constructor and Destructor Execution Order in Inheritance

When a derived object is created on the stack (`grad_student g;`):

1. **Constructors Execute from Top Base Down to Most-Derived (`Base → Intermediate → Derived`)**:
   - Why? Because the derived class constructor may depend on base class members already being initialized!
   - In Multiple Inheritance (`class C : public A, public B`), base constructors execute in the **exact left-to-right order of declaration in the class header** (`A()` first, then `B()` , then `C()`), **regardless** of the order listed in a member initializer list!
2. **Destructors Execute in Strict Reverse Order (`Derived → Intermediate → Base`)**:
   - Why? Because the derived destructor may still need base subobject data while cleaning up derived resources; once the derived part is destroyed, the base part is destroyed last.

---

3. Function Overriding in Inheritance

When a derived class defines a member function with the exact same name and parameter signature as a member function in its base class:

```cpp
class super {
public:
    void show() { cout << "super show" << endl; }
};

class sub : public super {
public:
    void show() { cout << "sub show" << endl; } // Overrides/hides super::show()
};
```

- Calling `sub obj; obj.show();` invokes `sub::show()` .
- To explicitly call the base version on `obj` , use the scope resolution operator: `obj.super::show();` (or inside `sub::show()` , write `super::show();`).

04 — Ambiguity in Multiple Inheritance, the Diamond Problem, and `virtual` Base Classes

1. Member Name Ambiguity in Multiple Inheritance

Suppose two independent base classes `B1` and `B2` both have a function named `show()`, and class `D` inherits from both:

```cpp
class B1 {
public:
    void show() { cout << "From B1" << endl; }
};

class B2 {
public:
    void show() { cout << "From B2" << endl; }
};

class D : public B1, public B2 {
};
```

If `main()` writes `D obj; obj.show();`, the compiler reports an **ambiguity error**: `request for member 'show' is ambiguous`.

- **Resolution**: Qualify the call using the scope resolution operator (`::`):

  ```cpp
  obj.B1::show();
  obj.B2::show();
  ```

2. The Diamond Problem (Multipath Inheritance Duplication)

Consider a 4-class diamond hierarchy:

- Top base class: `person` (contains `int age;`)
- Two intermediate classes: `student : public person` and `employee : public person`
- Bottom class: `teaching_assistant : public student, public employee`

What Goes Wrong Without `virtual` Inheritance?

An object of `teaching_assistant` contains **TWO separate** `person` **subobjects** in memory:

1. One `person::age` inherited via the `student` path (`ta.student::age`).
2. A second, duplicate `person::age` inherited via the `employee` path (`ta.employee::age`).

Writing `ta.age = 22;` fails to compile with `error: request for member 'age' is ambiguous`! Moreover, the `person` constructor runs **twice**.

---

3. Solution: `virtual` Base Classes (`virtual public`)

To ensure that `teaching_assistant` shares **only ONE single common** `person` **subobject**, the intermediate classes (`student` and `employee`) must inherit from `person` using the `virtual` keyword:

CPP

```cpp
class person {
public:
    int age;
};

class student : virtual public person { };
class employee : virtual public person { };

class teaching_assistant : public student, public employee { };
```

Rules of `virtual` Base Classes (`[class.mi]`)

1. `virtual public` (or `public virtual` — both orders are identical) guarantees that only **one shared instance** of the `person` base subobject exists inside `teaching_assistant`.
2. `ta.age = 22;` is now completely **unambiguous** and valid!
3. **Constructor Rule for Virtual Base Classes**: Virtual base class constructors are always invoked **before** any non-virtual base class constructors, and the **most-derived class** (`teaching_assistant`) is responsible for initializing the virtual base class (`person`).

Visual Concept & Memory Diagrams (10 INHERITANCE)

Inheritance Access Specifier Transformation Matrix

Base Member	: public Base	: protected Base	: private Base
public	public	protected	private
protected	protected	protected	private
private	Inaccessible	Inaccessible	Inaccessible

Diagram: diagram 01 access specifier transformation

Mandatory Classroom Examples 1 & 2: Inside Derived vs. Outside Derived (main)

Base: person / super

```
public:
  int input(int a)
  void show()
```

Derived: : protected Base

```
protected (Transformed!):
  input(int a)
  show()
----------------------------
public (Inside Derived):
  void display() {
    input(30); // ALLOWED!
  }
```

Outside: int main()

```
obj.show();
COMPILE ERROR!
s.input(40);
COMPILE ERROR!
----------------------
s.display();
ALLOWED (public)!
```

Diagram: diagram 02 classroom protected inheritance

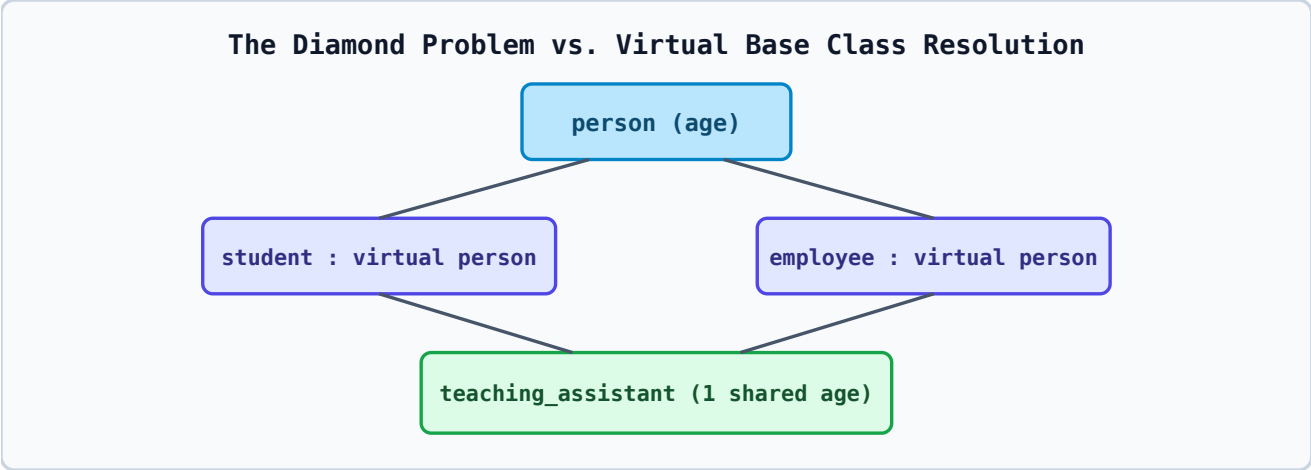


Diagram: diagram 03 inheritance types and diamond

Annotated C++ Programs (10 INHERITANCE)

Program: `01_single_public_inheritance.cpp`

C++ SOURCE FILE • 01_single_public_inheritance.cpp

```cpp
// Program Title: Simple Single Public Inheritance (person and student)
// Topic: Inheritance — Single Public Inheritance
// Purpose: Demonstrate how a derived class (student) inherits public members from a base
class (person).
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_inheritance_fundamentals_and_access_modes.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class person {
protected:
    int age;

public:
    void set_age(int a) {
        age = a;
    }
};

class student : public person {
private:
    int roll;

public:
    void set_roll(int r) {
        roll = r;
    }

    void display() {
        cout << "Roll: " << roll << ", Age: " << age << endl;
    }
};

int main() {
    student s;

    s.set_age(20);
    s.set_roll(101);
    s.display();

    return 0;
}

/*
Expected Deterministic Output:
```

```
Roll: 101, Age: 20
*/
```

Program: `02_classroom_super_sub_protected_compile_error.cpp`

C++ SOURCE FILE • 02_classroom_super_sub_protected_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Mandatory Classroom Example 1 — Protected Inheritance Access Violation (super and sub)
// Topic: Inheritance — Protected Inheritance Access Control
// Purpose: Preserve and explain the classroom program where obj.show() fails in main() due to protected inheritance.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/02_mandatory_classroom_examples_analysis.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

class super {
public:
    void show() {
        cout << "hello from super" << endl;
    }
};

class sub : protected super {
};

int main() {
    sub obj;
    obj.show(); // Intentional compilation error
    return 0;
}

/*
Why Compilation Fails:
1. 'show()' is declared 'public' inside 'class super'.
2. 'class sub : protected super' inherits from 'super' using 'protected' visibility.
3. Under C++ standard rule [class.access.base], all 'public' members of 'super' become 'protected' members of 'sub'.
4. 'main()' is an external function outside 'sub'. External functions cannot call 'protected' members through an object ('obj.show()').
See 03_classroom_super_sub_fixed_public.cpp and 04_classroom_super_sub_fixed_wrapper.cpp for the corrected Category A versions.
*/
```

Program: `03_classroom_super_sub_fixed_public.cpp`

C++ SOURCE FILE • 03_classroom_super_sub_fixed_public.cpp

```cpp
// Program Title: Corrected Classroom Example 1 (Fix A: Using Public Inheritance)
// Topic: Inheritance — Public Inheritance
// Purpose: Fix Classroom Example 1 by changing 'protected super' to 'public super' so
obj.show() is accessible in main().
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_mandatory_classroom_examples_analysis.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class super {
public:
    void show() {
        cout << "hello from super" << endl;
    }
};

class sub : public super {
};

int main() {
    sub obj;
    obj.show();
    return 0;
}

/*
Expected Deterministic Output:
hello from super
*/
```

Program: `04_classroom_super_sub_fixed_wrapper.cpp`

C++ SOURCE FILE • 04_classroom_super_sub_fixed_wrapper.cpp

```cpp
// Program Title: Corrected Classroom Example 1 (Fix B: Calling Protected Member from Inside sub)
// Topic: Inheritance — Protected Inheritance with Public Wrapper Member Function
// Purpose: Keep 'class sub : protected super' and demonstrate that sub's own member function can call show().
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_mandatory_classroom_examples_analysis.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class super {
public:
    void show() {
        cout << "hello from super" << endl;
    }
};

class sub : protected super {
public:
    void display() {
        show(); // Legal! Inside sub, the protected inherited member show() is accessible.
    }
};

int main() {
    sub obj;
    obj.display();
    return 0;
}

/*
Expected Deterministic Output:
hello from super
*/
```

Program: `05_classroom_person_student_compile_error.cpp`

C++ SOURCE FILE • 05_classroom_person_student_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Mandatory Classroom Example 2 — Inside-Derived vs. Outside-Derived Access (person and student)
// Topic: Inheritance — Protected Inheritance Access Transformation
// Purpose: Demonstrate why s.input(40) fails in main() while display() calling input(30) inside student is legal.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/02_mandatory_classroom_examples_analysis.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

class person {
public:
    int age;

    int input(int a) {
        age = a;
        return age;
    }
};

class student : protected person {
public:
    void display() {
        cout << input(30) << endl;
    }
};

int main() {
    student s;

    s.input(40); // Intentional compilation error

    s.display();
    return 0;
}

/*
Why Compilation Fails on 's.input(40)' While 'display()' Succeeds:
1. 'person::input(int)' is 'public' in 'person', but becomes 'protected' in 'student' due to ': protected person'.
2. Inside 'student::display()', calling 'input(30)' is LEGAL because a derived class member function can access its own 'protected' inherited members.
3. Inside 'main()', calling 's.input(40)' is ILLEGAL (Compile Error) because 'main()' is outside 'student' and cannot access 'protected' members of 'student'.
See 06_classroom_person_student_corrected.cpp for the corrected Category A version.
*/
```

Program: `06_classroom_person_student_corrected.cpp`

C++ SOURCE FILE • 06_classroom_person_student_corrected.cpp

```cpp
// Program Title: Corrected Classroom Example 2 — Accessing Protected Inherited input() via student::display()
// Topic: Inheritance — Protected Inheritance Access from Inside Derived Class
// Purpose: Demonstrate the working version of Classroom Example 2 where s.display() calls inherited input(30).
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_mandatory_classroom_examples_analysis.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class person {
public:
    int age;

    int input(int a) {
        age = a;
        return age;
    }
};

class student : protected person {
public:
    void display() {
        cout << input(30) << endl;
    }
};

int main() {
    student s;

    // s.input(40) is omitted here because input() is protected in student.
    // Instead, we call the public member function s.display():
    s.display();

    return 0;
}

/*
Expected Deterministic Output:
30
*/
```

Program: `07_multilevel_and_constructor_order.cpp`

C++ SOURCE FILE • 07_multilevel_and_constructor_order.cpp

```cpp
// Program Title: Multilevel Inheritance Constructor and Destructor Order
// Topic: Inheritance — Constructor and Destructor Invocation Sequence
// Purpose: Show that constructors run Base → Derived and destructors run Derived → Base.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/03_types_of_inheritance_and_constructor_order.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class person {
public:
    person() {
        cout << "1. person constructor" << endl;
    }
    ~person() {
        cout << "6. person destructor" << endl;
    }
};

class student : public person {
public:
    student() {
        cout << "2. student constructor" << endl;
    }
    ~student() {
        cout << "5. student destructor" << endl;
    }
};

class result : public student {
public:
    result() {
        cout << "3. result constructor" << endl;
    }
    ~result() {
        cout << "4. result destructor" << endl;
    }
};

int main() {
    result r;
    return 0;
}

/*
Expected Deterministic Output:
1. person constructor
2. student constructor
3. result constructor
4. result destructor
5. student destructor
```

```
6. person destructor
*/
```

Program: `08_diamond_problem_virtual_base.cpp`

C++ SOURCE FILE • 08_diamond_problem_virtual_base.cpp

```cpp
// Program Title: Solving the Diamond Problem Using Virtual Base Classes
// Topic: Inheritance — Multipath Inheritance and virtual public Base Classes
// Purpose: Demonstrate unambiguous access to a single shared 'person' subobject via virtual inheritance.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/04_ambiguity_diamond_problem_and_virtual_base.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class person {
public:
    int age;

    void show_age() {
        cout << "Shared person age: " << age << endl;
    }
};

class student : virtual public person {
public:
    int roll;
};

class employee : virtual public person {
public:
    int salary;
};

class teaching_assistant : public student, public employee {
public:
    void display() {
        show_age();
        cout << "Roll: " << roll << ", Salary: " << salary << endl;
    }
};

int main() {
    teaching_assistant ta;

    ta.age = 23; // Unambiguous because student and employee inherit 'person' virtually!
    ta.roll = 105;
    ta.salary = 15000;
    ta.display();

    return 0;
}

/*
Expected Deterministic Output:
Shared person age: 23
```

```
Roll: 105, Salary: 15000
*/
```

Conceptual Questions — Module 10: Inheritance

Level 1 — Beginner

Q1. What is inheritance in C++? What is the default inheritance visibility mode when deriving a `class` vs. deriving a `struct`?

Q2. Draw the complete access-specifier transformation table for `public`, `protected`, and `private` base members when inherited via `public`, `protected`, and `private` modes.

Level 2 — Intermediate

Q3. In Classroom Example 1 (`class sub : protected super`), why does `sub obj; obj.show();` fail to compile in `main()` even though `show()` was declared `public` in `super`?

Q4. In Classroom Example 2 (`class student : protected person`), explain why `display()` inside `student` can call `input(30)`, whereas `s.input(40)` inside `main()` causes a compilation error.

Level 3 — Advanced

Q5. What is the exact order of constructor and destructor execution in a multilevel inheritance hierarchy (`A → B → C`) and in a multiple inheritance hierarchy (`class C : public A, public B`)?

Level 4 — University Examination Level

Q6. Explain the **Diamond Problem** in multipath inheritance with a diagram. How does `virtual` inheritance solve subobject duplication, and which class is responsible for invoking the constructor of a `virtual` base class?

Debugging Questions — Module 10: Inheritance

Q10. Identify why the following classroom program fails to compile and show how to fix it without making `age` public in `person`:

CPP

```cpp
class person {
private:
    int age;
};

class student : public person {
public:
    void set(int a) {
        age = a; // Error here
    }
};
```

Q11. Identify why calling `obj.greet()` is ambiguous in the following multiple inheritance program and provide the exact fix in `main()`:

CPP

```cpp
#include <iostream>
using namespace std;

class father {
public:
    void greet() { cout << "Hello from father\n"; }
};

class mother {
public:
    void greet() { cout << "Hello from mother\n"; }
};

class child : public father, public mother {
};

int main() {
    child obj;
    obj.greet();
    return 0;
}
```

Predict the Output Questions — Module 10: Inheritance

Level 1 — Beginner

Q7. Predict the output:

CPP

```cpp
#include <iostream>
using namespace std;

class base {
public:
    void show() { cout << "Base "; }
};

class derived : public base {
public:
    void show() { cout << "Derived "; }
};

int main() {
    derived d;
    d.show();
    d.base::show();
    cout << endl;
    return 0;
}
```

Level 2 — Intermediate

Q8. Predict the output:

CPP

```cpp
#include <iostream>
using namespace std;

class A {
public:
    A() { cout << "A "; }
    ~A() { cout << "~A "; }
};

class B {
public:
    B() { cout << "B "; }
    ~B() { cout << "~B "; }
};

class C : public B, public A {
public:
    C() { cout << "C "; }
    ~C() { cout << "~C "; }
};

int main() {
    C obj;
    return 0;
}
```

Level 3 — Advanced

Q9. Predict the output (Virtual base constructor order):

CPP

```cpp
#include <iostream>
using namespace std;

class V {
public:
    V() { cout << "V "; }
};

class A : public V {
public:
    A() { cout << "A "; }
};

class B : virtual public V {
public:
    B() { cout << "B "; }
};

class D : public A, public B {
public:
    D() { cout << "D "; }
};

int main() {
    D obj;
    cout << endl;
    return 0;
}
```

(Hint: Note carefully that `B` inherits `V` virtually, while `A` inherits `V` non-virtually!)

Programming Questions — Module 10: Inheritance

Q12. Write a complete C++23 classroom-style program demonstrating **Multilevel Inheritance**:

- Class `student` holds `roll` with `set_roll(int)` and `show_roll()`.
- Class `test : public student` holds `marks1` and `marks2` with `set_marks(int, int)` and `show_marks()`.
- Class `result : public test` computes `total = marks1 + marks2` and displays roll, marks, and total.

Q13. Write a complete C++23 classroom-style program demonstrating **Hybrid / Diamond Inheritance** using `virtual public`:

- Base class `student` (`roll`).
- Derived class `test : virtual public student` (`academic_marks`).
- Derived class `sports : virtual public student` (`sports_score`).
- Derived class `result : public test, public sports` that sums `academic_marks + sports_score` and prints the complete scorecard.

Detailed Solutions — Conceptual Questions (Module 10: Inheritance)

Solution to Q1 (Default Inheritance Modes)

- **Concept**: Base-specifier defaults (`[class.access.base]`).
- **Rule**: When deriving a `class` (`class D : B`), the default inheritance mode is `private` . When deriving a `struct` (`struct D : B`), the default inheritance mode is `public` .

Solution to Q2 (Access Transformation Matrix)

- `public` base members become `public` under `public` inheritance, `protected` under `protected` inheritance, and `private` under `private` inheritance.
- `protected` base members become `protected` under `public` and `protected` inheritance, and `private` under `private` inheritance.
- `private` base members are never directly accessible inside the derived class under any inheritance mode.

Solution to Q3 (Classroom Example 1: `super` and `sub`)

- **Rule**: In `class sub : protected super` , `super::show()` (originally `public` in `super`) becomes a **`protected` member of `sub`** . Because `main()` is outside `class sub` and is not a derived class or friend of `sub` , calling `obj.show()` in `main()` violates C++ access control and fails to compile.

Solution to Q4 (Classroom Example 2: `person` and `student`)

- **Rule**: In `class student : protected person` , `person::input(int)` becomes `protected` in `student` .
  - Inside `student::display()` , the code is executing inside a member function of `student` , where `protected` members of `student` are **accessible** (`input(30)` is valid).
  - Inside `main()` , `s.input(40)` is an outside access through an object `s` of type `student` . Outside code cannot access `protected` members (`s.input(40)` is a compilation error).

Solution to Q5 (Constructor and Destructor Order)

- **Multilevel (`A → B → C`)**: Constructors run `A()` , then `B()` , then `C()` . Destructors run `~C()` , then `~B()` , then `~A()` .
- **Multiple (`class C : public A, public B`)**: Base constructors execute in declaration order (`A()` , then `B()` , then `C()`), and destructors execute in exact reverse order (`~C()` , then `~B()` , then `~A()`).

Solution to Q6 (Diamond Problem & Virtual Base Classes)

- Without `virtual` inheritance, `teaching_assistant` inherits two separate copies of `person` (one through `student` and one through `employee`), causing ambiguity when accessing `age` .
- Declaring `class student : virtual public person` and `class employee : virtual public person` merges the two `person` subobjects into a single shared virtual base subobject. The **most-**

derived class (`teaching_assistant`) initializes the virtual base class `person` before any non-virtual base constructors run.

Detailed Solutions — Predict the Output & Debugging (Module 10: Inheritance)

Solution to Q7

- `d.show()` calls `derived::show()` (`Derived`).
- `d.base::show()` explicitly calls `base::show()` via the scope resolution operator (`Base`).
- **Guaranteed Output**:

```
TEXT
Derived Base
```

Solution to Q8

- In `class C : public B, public A` , `B` is listed before `A` in the class header.
- Therefore, constructors run `B()` , then `A()` , then `C()` .
- Destructors run in reverse: `~C()` , then `~A()` , then `~B()` .
- **Guaranteed Output**:

```
TEXT
B A C ~C ~A ~B
```

Solution to Q9

- **Rule (`[class.base.init]`)**: Virtual base classes are always initialized **first**, before any non-virtual base classes!
- Step 1: `B` has a virtual base `V` , so the shared virtual base `V()` is constructed first (`V`).
- Step 2: Non-virtual bases of `D` are initialized in declaration order (`A` , then `B`).
  - `A` inherits `V` *non-virtually*, so `A` 's own separate `V` subobject is constructed (`V`), followed by `A()` (`A`).
  - `B` 's virtual base `V` was already constructed in Step 1, so `B()` runs (`B`).
- Step 3: `D()` runs (`D`).
- **Guaranteed Output**:

```
TEXT
V V A B D
```

Solution to Q10 (Debugging `private` Base Member)

- **Error**: `age` is `private` in `person` , so `student::set(int)` cannot access `age` directly even with `public` inheritance.

- **Fix**: Change `private:` to `protected:` in `class person` (or provide a `public: void set_age(int)` member function in `person`).

Solution to Q11 (Debugging Multiple Inheritance Ambiguity)

- **Error**: Both `father` and `mother` define `void greet()`, so `obj.greet()` is ambiguous.
- **Fix**: Disambiguate using the scope resolution operator in `main()`:

```cpp
obj.father::greet();
obj.mother::greet();
```

Detailed Solutions — Programming Questions (Module 10:

Inheritance)

Solution to Q12 (Multilevel Inheritance: `student → test → result`)

CPP

```cpp
#include <iostream>
using namespace std;

class student {
protected:
    int roll;

public:
    void set_roll(int r) {
        roll = r;
    }

    void show_roll() {
        cout << "Roll No: " << roll << endl;
    }
};

class test : public student {
protected:
    int marks1;
    int marks2;

public:
    void set_marks(int m1, int m2) {
        marks1 = m1;
        marks2 = m2;
    }

    void show_marks() {
        cout << "Marks: " << marks1 << ", " << marks2 << endl;
    }
};

class result : public test {
private:
    int total;

public:
    void display() {
        total = marks1 + marks2;
        show_roll();
        show_marks();
        cout << "Total Marks: " << total << endl;
    }
};

int main() {
    result r;
    r.set_roll(101);
    r.set_marks(88, 92);
```

```
    r.display();
    return 0;
}
/*
Expected Output:
Roll No: 101
Marks: 88, 92
Total Marks: 180
*/
```

Solution to Q13 (Hybrid Diamond Inheritance with `virtual public`)

CPP

```cpp
#include <iostream>
using namespace std;

class student {
protected:
    int roll;

public:
    void set_roll(int r) {
        roll = r;
    }

    void show_roll() {
        cout << "Roll No: " << roll << endl;
    }
};

class test : virtual public student {
protected:
    int academic_marks;

public:
    void set_academic(int m) {
        academic_marks = m;
    }
};

class sports : virtual public student {
protected:
    int sports_score;

public:
    void set_sports(int s) {
        sports_score = s;
    }
};

class result : public test, public sports {
public:
    void display() {
        int total = academic_marks + sports_score;
        show_roll();
        cout << "Academic: " << academic_marks
             << ", Sports: " << sports_score
             << ", Final Total: " << total << endl;
    }
};

int main() {
    result res;
    res.set_roll(202);
    res.set_academic(85);
    res.set_sports(10);
```

```
    res.display();
    return 0;
}
/*
Expected Output:
Roll No: 202
Academic: 85, Sports: 10, Final Total: 95
*/
```